WEEKLY ASSESSMENT - 6

1. Write a recursive function for the Fibonacci sequence.

A recursive Fibonacci function calculates the n-th Fibonacci number by adding the two preceding terms, using base cases for 0 and 1:

```
def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Example usage:
print(fibonacci(7))  # Output: 13
```

2. Explain "Memoization" and how it helps recursive functions.

Memoization is an optimization technique that speeds up programs by storing the results of expensive function calls and returning the cached result when the same inputs occur again. It avoids redundant computations in recursive functions (like overlapping subproblems in naive Fibonacci recursion), transforming exponential time complexity $O(2^n)$ into linear time complexity $O(n)$.

```
# Python manual memoization using a dictionary cache
cache = {}
def fib_memo(n):
    if n in cache:
        return cache[n]
    if n <= 1:
        return n
    cache[n] = fib_memo(n - 1) + fib_memo(n - 2)
    return cache[n]
```

3. Use reduce() from functools to find the product of a list.

The `reduce()` function applies a binary function continually to the elements of a sequence, reducing the entire list to a single cumulative value:

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)

print(product)  # Output: 120
```

4. Write a function to check if a number is an "Armstrong Number."

An Armstrong number is a number that is equal to the sum of its own digits each raised to the power of the number of digits:

```
def is_armstrong(num):
    digits = [int(d) for d in str(num)]
    power = len(digits)
    return sum(d ** power for d in digits) == num

print(is_armstrong(153))  # Output: True (1^3 + 5^3 + 3^3 = 153)
print(is_armstrong(9474)) # Output: True (9^4 + 4^4 + 7^4 + 4^4 =
9474)
```

5. Create a closure that remembers a base value and adds it to an input.

A closure occurs when a nested function references variables from its enclosing outer scope, retaining access to those variables even after the outer function finishes executing:

```
def make_adder(base_value):
    def adder(input_value):
        return base_value + input_value
    return adder

add_five = make_adder(5)
print(add_five(10))  # Output: 15
print(add_five(20))  # Output: 25
```

6. Explain the "LEGB" rule in Python scope.

The **LEGB** rule defines the specific sequence order Python uses to search for variable names in scope contexts:

1. **Local (L):** Names assigned in any way within a function body and not declared global.
2. **Enclosing (E):** Names in the local scope of any and all enclosing functions (nested scopes).
3. **Global (G):** Names assigned at the top-level of a module file, or declared global in a def.
4. **Built-in (B):** Names preassigned in Python's built-in names module (e.g., `print`, `len`).

7. Write a function that converts a decimal number to binary without bin().

Decimal-to-binary conversion can be handled via standard iterative division operations processing remainders:

```
def decimal_to_binary(n):
    if n == 0:
        return "0"
    binary_str = ""
```

```
    while n > 0:
        binary_str = str(n % 2) + binary_str
        n = n // 2
    return binary_str

print(decimal_to_binary(10))  # Output: 1010
print(decimal_to_binary(25))  # Output: 11001
```

8. Create a "Generator" function using yield to produce infinite even numbers.

Generators use the `yield` keyword to suspend state dynamically, creating custom iterators that construct stream pipelines on-demand without overhead:

```
def infinite_evens():
    num = 0
    while True:
        yield num
        num += 2

gen = infinite_evens()
print(next(gen))  # Output: 0
print(next(gen))  # Output: 2
print(next(gen))  # Output: 4
```

9. How do you pass a function as an argument to another function?

Functions in Python are **first-class citizens**, meaning they can be bound to references, treated as elements, and cleanly injected into secondary function parameters:

```
def formal_greeting(name):
    return f"Hello, {name}."

def process_user(greeting_func, name):
    return greeting_func(name)

# Passing 'formal_greeting' reference without calling brackets
result = process_user(formal_greeting, "Ajay")
print(result)  # Output: Hello, Ajay.
```

10. Write a function to validate a password (min 8 chars, 1 digit, 1 symbol).

This routine checks string validation rules manually using Python's primitive character analysis helper routines:

```
def validate_password(password):
    if len(password) < 8:
        return False
```

```
    has_digit = any(char.isdigit() for char in password)

    # A character is considered a symbol if it is not alphanumeric
    has_symbol = any(not char.isalnum() for char in password)

    return has_digit and has_symbol

print(validate_password("SecurePassword1!"))  # Output: True
print(validate_password("short1!"))           # Output: False (Under
8 chars)
```